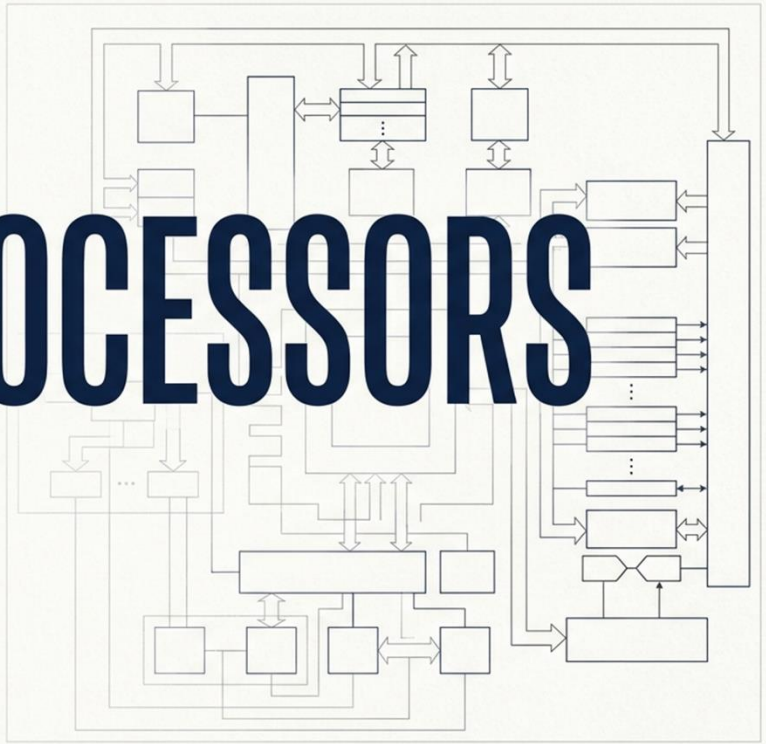
MICROPROCESSORS

CHAPTER 1

SEN. INST. LEILA VAIGHAN



Chapter Objectives

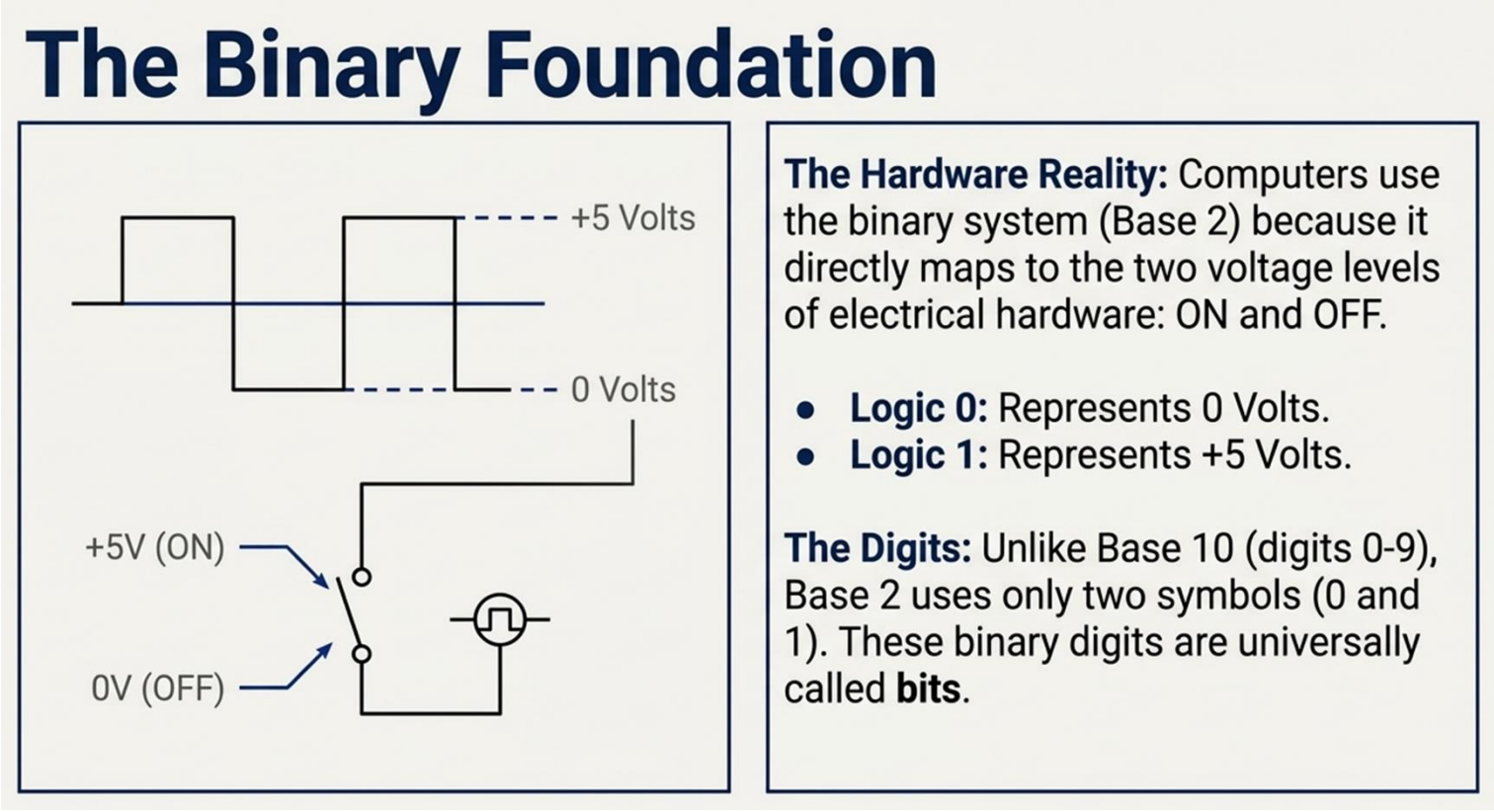
1.	Convert numbers between base 2, base 10, and base 16.
2.	Describe logic operations: AND, OR, NOT, XOR, NAND, and NOR.
3.	Differentiate between a bit, nibble, byte, and word.
4.	Define kilobyte, megabyte, gigabyte, and terabyte mathematically.
5.	Describe the purpose of major computer system components and CPU architecture.

Introduction to Computing

SECTION 1: NUMBERING AND CODING SYSTEMS

Decimal and binary number systems

Although there has been speculation that the origin of the base 10 system is the fact that human beings have 10 fingers, there is absolutely no speculation about the reason behind the use of the binary system in computers. The binary system is used in computers because 1 and 0 represent the two voltage levels of on and off.
Whereas in base 10 there are 10 distinct symbols, 0, 1, 2, ..., 9, in base 2 there are only two, 0 and 1, with which to generate numbers. Base 10 contains digits 0 through 9; binary contains digits 0 and 1 only. These two binary digits, 0 and 1, are commonly referred to as *bits*.



Converting from decimal to binary

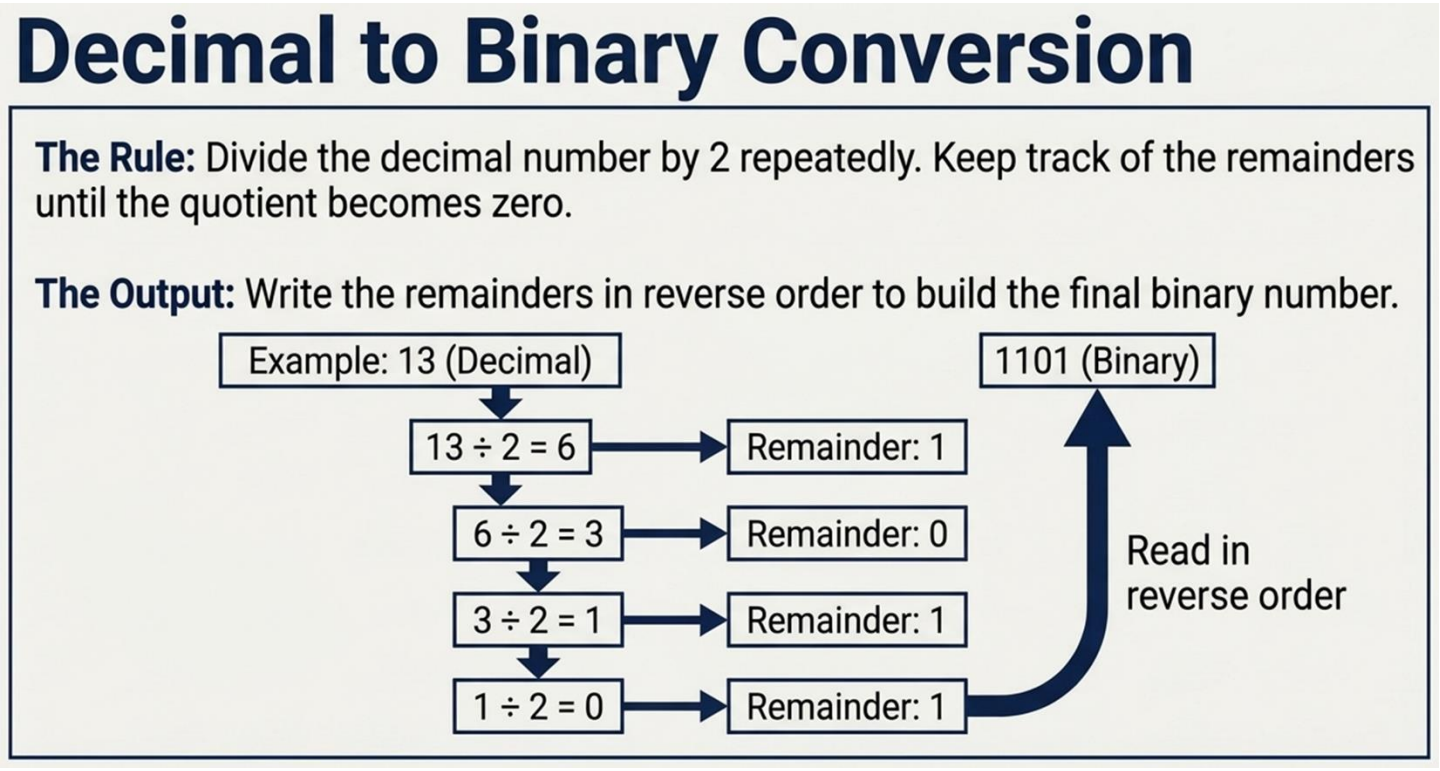
One method of converting from decimal to binary is to divide the decimal number by 2 repeatedly, keeping track of the remainders. This process continues until the quotient becomes zero.
The remainders are then written in reverse order to obtain the binary number.

Convert 25_{10} to binary.

Solution:

	Quotient	Remainder	
25/2 =	12	1	LSB (least significant bit)
12/2 =	6	0	
6/2 =	3	0	
3/2 =	1	1	
1/2 =	0	1	MSB (most significant bit)

Therefore, $25_{10} = 11001_2$.



Converting from binary to decimal

To convert from binary to decimal, it is important to understand the concept of weight associated with each digit position. First, as an analogy, recall the weight of numbers in the base 10 system. By the same token, each digit position of a number in base 2 has a weight associated with it:

110101_2 =				***Decimal***	***Binary***
1×2^0	=	1×1	=	1	1
0×2^1	=	0×2	=	0	00
1×2^2	=	1×4	=	4	100
0×2^3	=	0×8	=	0	0000
1×2^4	=	1×16	=	16	10000
1×2^5	=	1×32	=	32	100000
				53	110101

Knowing the weight of each bit in a binary number makes it simple to add them together to get its decimal equivalent.

Convert 11001_2 to decimal.

Solution:

Weight:	16	8	4	2	1
Digits:	1	1	0	0	1
Sum:	16 +	8 +	0 +	0 +	$1 = 25_{10}$

Knowing the weight associated with each binary bit position allows one to convert a decimal number to binary directly instead of going through the process of repeated division.

Use the concept of weight to convert 39_{10} to binary.

Solution:

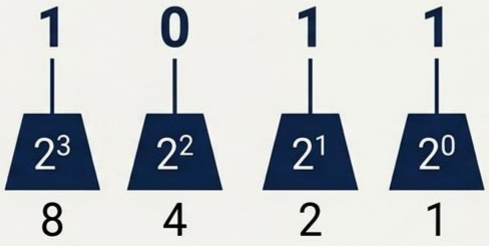
Weight:	32	16	8	4	2	1
	1	0	0	1	1	1
	32 +	0 +	0 +	4 +	2 +	1 = 39

Therefore, $39_{10} = 100111_2$.

Binary to Decimal Conversion

The Weight Concept: Just like the Base 10 system, every digit position in a Base 2 number carries a specific mathematical weight.

The Rule: Determine the weight of each bit position. Add together the weights of all positions that contain a '1' to find the decimal equivalent.



$$8 + 4 + 2 + 1 = \mathbf{11}$$

Hexadecimal system

Base 16, or the *hexadecimal* system as it is called in computer literature, is used as a convenient representation of binary numbers. For example, it is much easier for a human being to represent a string of 0s and 1s such as 100010010110 as its hexadecimal equivalent of 896H. The binary system has 2 digits, 0 and 1. The base 10 system has 10 digits, 0 through 9. The hexadecimal (base 16) system has 16 digits. In base 16, the first 10 digits, 0 to 9, are the same as in decimal, and for the remaining six digits, the letters A, B, C, D, E, and F are used.

Decimal	Binary	Hex
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

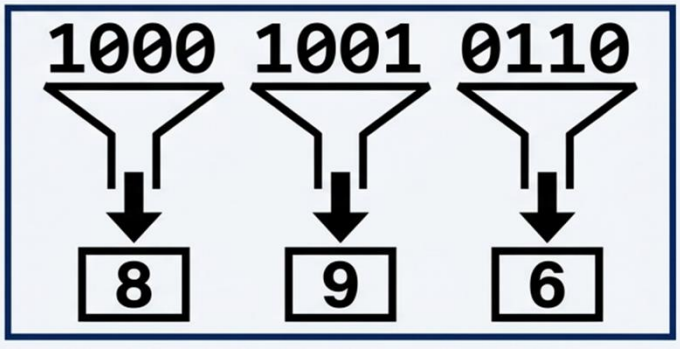
The Hexadecimal System

The Purpose: Base 16 (Hexadecimal) is used in computer literature as a convenient, shortened representation of binary numbers.

Example: It is easier for humans to read 896H than the binary equivalent 100010010110.

The 16 Digits: Uses the standard digits 0 to 9, then transitions to letters A, B, C, D, E, and F for the remaining six values.

0	1	2	3
4	5	6	7
8	9	A	B
C	D	E	F



Converting between binary and hex

To represent a binary number as its equivalent hexadecimal number, start from the right and group 4 bits at a time, replacing each 4-bit binary number with its hex equivalent.
To convert from hex to binary, each hex digit is replaced with its 4-bit binary equivalent.

Represent binary 100111110101 in hex.

Solution:
First the number is grouped into sets of 4 bits: 1001 1111 0101.
Then each group of 4 bits is replaced with its hex equivalent:

1001	1111	0101
9	F	5

Therefore, 100111110101_2 = 9F5 hexadecimal.

Convert hex 29B to binary.

Solution:

		2	9	B
29B	=	0010	1001	1011

Dropping the leading zeros gives 1010011011.

Converting from decimal to hex

Converting from decimal to hex could be approached in two ways:

1. Convert to binary first and then convert to hex.
2. Convert directly from decimal to hex by repeated division, keeping track of the remainders.

(a) Convert 45_{10} to hex.

32	16	8	4	2	1	First, convert to binary.
1	0	1	1	0	1	32 + 8 + 4 + 1 = 45

$45_{10} = 0010\ 1101_2 = 2D$ hex

(b) Convert 629_{10} to hex.

512	256	128	64	32	16	8	4	2	1
1	0	0	1	1	1	0	1	0	1

$629_{10} = (512 + 64 + 32 + 16 + 4 + 1) = 0010\ 0111\ 0101_2 = 275$ hex

(c) Convert 1714_{10} to hex.

1024	512	256	128	64	32	16	8	4	2	1
1	1	0	1	0	1	1	0	0	1	0

$1714_{10} = (1024 + 512 + 128 + 32 + 16 + 2) = 0110\ 1011\ 0010_2 = 6B2$ hex

Converting from hex to decimal

Conversion from hex to decimal can also be approached in two ways:

1. Convert from hex to binary and then to decimal.
2. Convert directly from hex to decimal by summing the weight of all digits.

Convert the following hexadecimal numbers to decimal.

(a) $6B2_{16} = 0110\ 1011\ 0010_2$

1024	512	256	128	64	32	16	8	4	2	1
1	1	0	1	0	1	1	0	0	1	0

$1024 + 512 + 128 + 32 + 16 + 2 = 1714_{10}$

(b) $9F2D_{16} = 1001\ 1111\ 0010\ 1101_2$

32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1
1	0	0	1	1	1	1	1	0	0	1	0	1	1	0	1

$32768 + 4096 + 2048 + 1024 + 512 + 256 + 32 + 8 + 4 + 1 = 40{,}749_{10}$

Addition of binary numbers

A + B	Carry	Sum
0 + 0	0	0
0 + 1	0	1
1 + 0	0	1
1 + 1	1	0

Add the following binary numbers. Check against their decimal equivalents.

Solution:

	Binary	*Decimal*
	1101	13
+	1001	9
	10110	22

Subtraction of binary numbers

Given binary numbers x and y.

x–y is performed by taking 2's complement of y and adding to x.

Example: 11001001 – 10011101

=> 2's comp. of 10011101= 01100010 (1's comp.)

+ 1

01100011 (2's comp.)

11001001

+ 01100011

1 00101100 (ignore the carry) 11001001 – 10011101 = 00101100

2's complement

To get the 2's complement of a binary number, invert all the bits and then add 1 to the result.

Take the 2's complement of 10011101.

Solution:

	10011101	binary number
	01100010	1's complement
+	1	
	01100011	2's complement

Addition and Subtraction of Hex numbers

Addition Example :

23D9	LSD : 9 + 14= 23	23-16= **7** with a carry to next digit
+ 94BE	1+13+11= 25	25-16= **9** with a carry to next digit
B897	1+3+4=	**8**
	MSD 2 + 9=	**B**

Subtraction Example :

59F	LSD : 15 - 8	= **7**
– 2B8	25 (9+16) – 11 (B)	= 14, which is **E**
2E7	MSD 4 (5–1) –2	= **2**

ASCII code

The discussion so far has revolved around the representation of number systems. Because all information in the computer must be represented by 0s and 1s, binary patterns must be assigned to letters and other characters. In the 1960s a standard representation called *ASCII* (American Standard Code for Information Interchange) was established.
The ASCII (pronounced “ask-E”) code assigns binary patterns for numbers 0 to 9, all the letters of the English alphabet, both uppercase (capital) and lowercase, and many control codes and punctuation marks. The great advantage of this system is that it is used by most computers, so that information can be shared among computers.
The ASCII system uses a total of 7 bits to represent each code.
For example, 100 0001 is assigned to the uppercase letter “A” and 110 0001 is for the lowercase “a”.

Often, a zero is placed in the most-significant bit position to make it an 8-bit code.

The use of ASCII is not only standard for keyboards used in the United States and many other countries but also provides a standard for printing and displaying characters by output devices such as printers and monitors.

ASCII Code : (American Standard Code for Information Interchange)

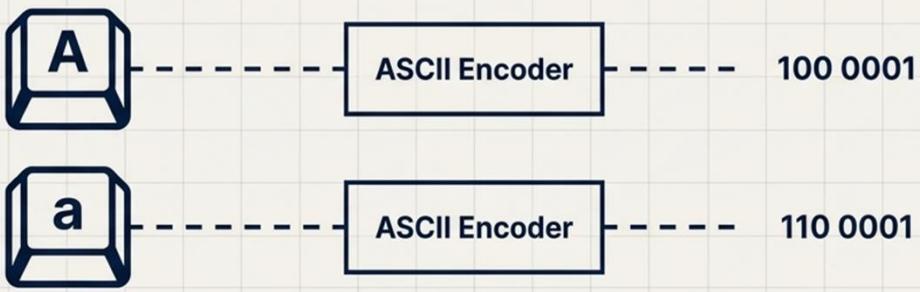
Hex	Symbol	Hex	Symbol
41	A	61	a
42	B	62	b
43	C	63	c
44	D	64	d
45	E	65	e
46	F	66	f
47	G	67	g
48	H	68	h
49	I	69	i
4A	J	6A	j
4B	K	6B	k
4C	L	6C	l
4D	M	6D	m
4E	N	6E	n
4F	O	6F	o
50	P	70	p
51	Q	71	q
52	R	72	r
53	S	73	s
54	T	74	t
55	U	75	u
56	V	76	v
57	W	77	w
58	X	78	x
59	Y	79	y
5A	Z	7A	z

The ASCII Standard

The Problem:
Computers only understand 0s and 1s. Letters and characters require standardized binary patterns.

The Solution:
ASCII (American Standard Code for Information Interchange) was created in the 1960s to assign binary codes to letters, numbers (0-9), and punctuation.

Structure:
Uses a 7-bit code. Often padded with a leading zero to create an 8-bit code.



SECTION 2: DIGITAL PRIMER

Binary logic

As mentioned earlier, computers use the binary number system because the two voltage levels can be represented as the two digits 0 and 1. Signals in digital electronics have two distinct voltage levels. For example, a system may define 0 V as logic 0 and +5 V as logic 1.

Logic gates

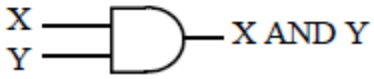
Binary logic gates are simple circuits that take one or more input signals and send out one output signal.

AND gate

The AND gate takes two or more inputs and performs a logic AND on them. See the truth table and diagram of the AND gate. Notice that if both inputs to the AND gate are 1, the output will be 1. Any other combination of inputs will give a 0 output. The example shows two inputs, *x* and *y*. Multiple outputs are also possible for logic gates. In the case of AND, if all inputs are 1, the output is 1. If any input is 0, the output is 0.

Logical AND Function

Inputs	Output
X Y	X AND Y
0 0	0
0 1	0
1 0	0
1 1	1

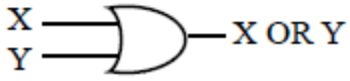


OR gate

The OR logic function will output a 1 if one or more inputs is 1. If all inputs are 0, then and only then will the output be 0.

Logical OR Function

Inputs	Output
X Y	X OR Y
0 0	0
0 1	1
1 0	1
1 1	1

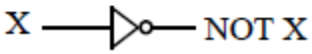


Inverter

The inverter, also called NOT, outputs the value opposite to that input to the gate. That is, a 1 input will give a 0 output, while a 0 input will give a 1 output.

Logical Inverter

Input	Output
X	NOT X
0	1
1	0

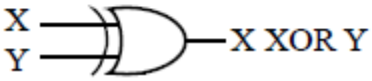


XOR gate

The XOR gate performs an exclusive-OR operation on the inputs. Exclusive-OR produces a 1 output if one (but only one) input is 1. If both operands are 0, the output is 0. Likewise, if both operands are 1, the output is also 0. Notice from the XOR truth table, that whenever the two inputs are the same, the output is 0. This function can be used to compare two bits to see if they are the same.

Logical XOR Function

Inputs	Output
X Y	X XOR Y
0 0	0
0 1	1
1 0	1
1 1	0

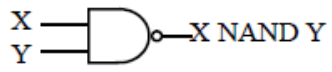


NAND and NOR gates

The NAND gate functions like an AND gate with an inverter on the output. It produces a 0 output when all inputs are 1; otherwise, it produces a 1 output. The NOR gate functions like an OR gate with an inverter on the output. It produces a 1 if all inputs are 0; otherwise, it produces a 0. NAND and NOR gates are used extensively in digital design because they are easy and inexpensive to fabricate. Any circuit that can be designed with AND, OR, XOR, and INVERTER gates can be implemented using only NAND and NOR gates. A simple example of this is given below. Notice in NAND, that if any input is 0, the output is 1. Notice in NOR, that if any input is 1, the output is 0.

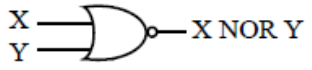
Logical NAND Function

Inputs	Output
X Y	X NAND Y
0 0	1
0 1	1
1 0	1
1 1	0



Logical NOR Function

Inputs	Output
X Y	X NOR Y
0 0	1
0 1	0
1 0	0
1 1	0



Decoder

A decoder is a combinational logic circuit that converts binary input data into a specific output.

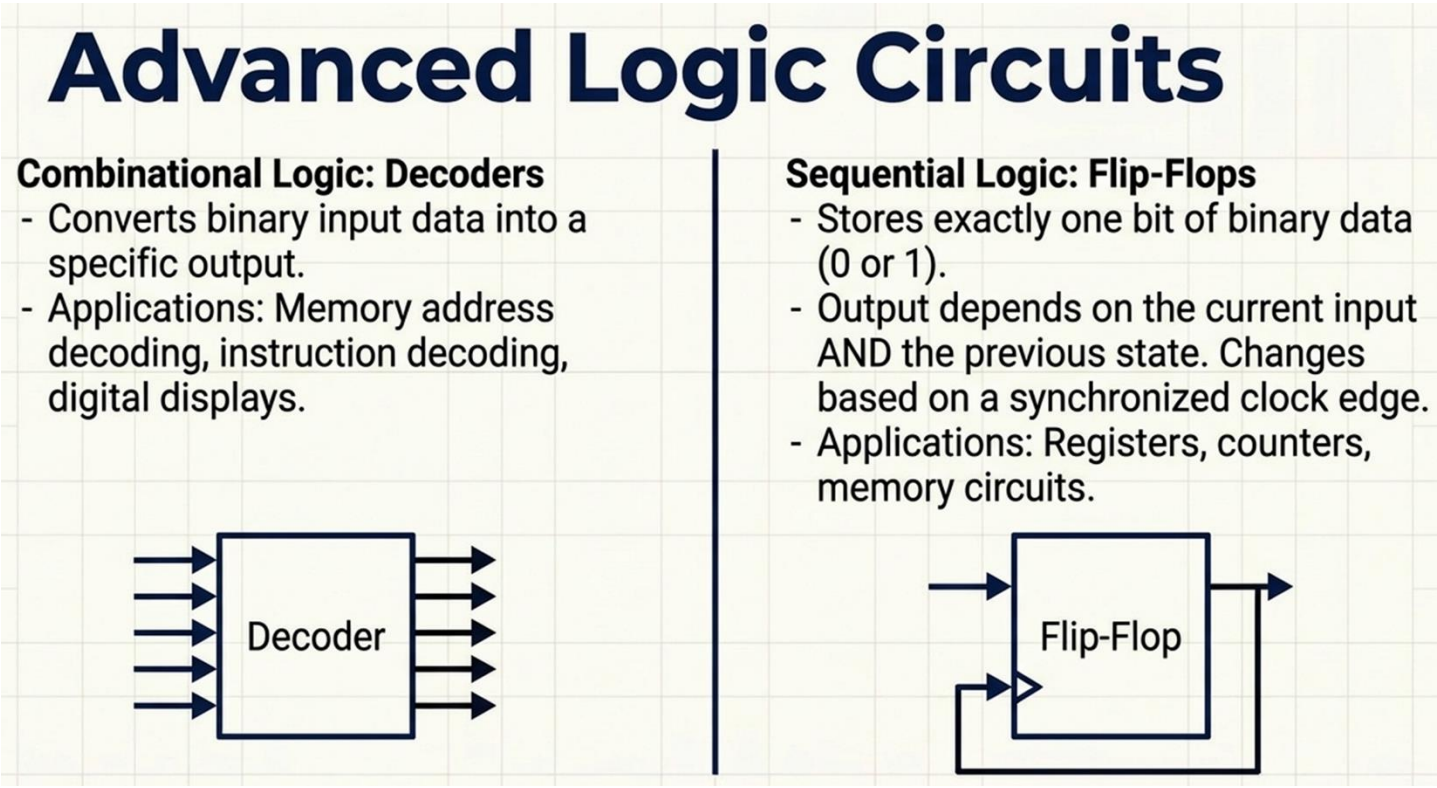
Applications: Memory address decoding, instruction decoding, and digital display systems.

Flip-Flop

A flip-flop is a sequential logic circuit that stores one bit of binary data (0 or 1). Unlike combinational circuits, its output depends on both the current input and its previous state.

Flip-flops commonly change their output in response to a triggering clock edge, allowing data to be stored and synchronized in digital systems.

Applications: Registers, counters, memory circuits, and frequency dividers.



Some important terminology

Bit	0
Nibble (4 bits)	0000
Byte (8 bits)	0000 0000
Word (16 bits)	0000 0000 0000 0000
double-word (32 bits)	0000 0000 0000 0000 0000 0000 0000 0000
quad-word (64 bits)	0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000

1 kilobyte (KB) is 2^{10} bytes - **1,024 bytes**
1 megabyte (MB) is 2^{20} bytes - **1,048,576 bytes** (a little over 1 million)
1 gigabyte (GB) is 2^{30} bytes - **1,073,741,824 bytes** (over 1 billion)
1 terabyte (TB) is 2^{40} bytes - **1,099,511,627,776 bytes** (over trillion)
1 petabyte (PB) is 2^{50} bytes – **1,125,899,906,842,624** bytes (over quadrillion)

Two types of memory commonly used in microcomputers:

1. RAM (Random Access Memory)

RAM, also known as read/write memory, is used for the temporary storage of programs and data currently being used by the computer.

- Volatile memory: Data is lost when the power is turned off.
- Read and write: Data can be read from and written to RAM.
- Purpose: Stores programs and data during computer operation.

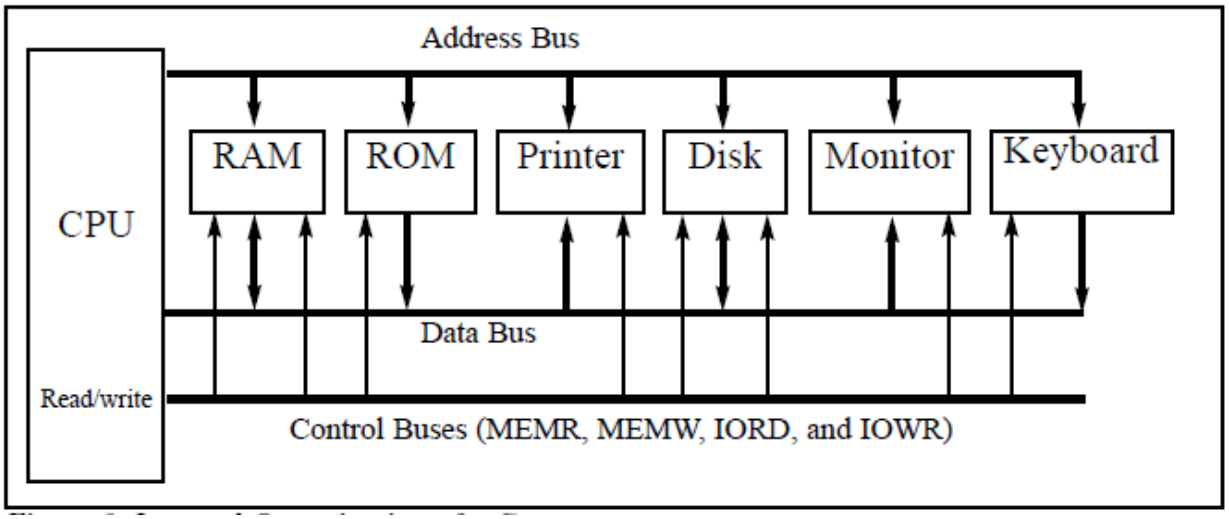
2. ROM (Read-Only Memory)

ROM is used to store programs and information essential to the operation of the computer.

- Nonvolatile memory: Data is retained when the power is turned off.
- Read-only: Traditionally, its contents cannot be changed by the user during normal operation.
- Purpose: Stores essential instructions required for computer operation.

Internal organization of computers

The internal working of every computer can be broken down into three parts: CPU (central processing unit), memory, and I/O (input/output) devices.



The function of the CPU is to execute (process) information stored in memory. The function of I/O devices such as the keyboard and video monitor is to provide a means of communicating with the CPU. The CPU is connected to memory and I/O through strips of wire called a *bus*. The bus inside a computer allows carrying information from place to place just as a street allows cars to carry people from place to place. In every computer there are three types of buses: address bus, data bus, and control bus. For a device (memory or I/O) to be recognized by the CPU, it must be assigned an address. The address assigned to a given device must be unique; no two devices are allowed to have the same address. The CPU puts the address (in binary, of course) on the address bus, and the decoding circuitry finds the device. Then the CPU uses the data bus either to get data from that device or to send data to it. The control buses are used to provide read or write signals to the device to indicate if the CPU is asking for information or sending information.

1. Data Bus

The data bus transfers data between the CPU, memory, and input/output devices. It is typically bidirectional, allowing data to move in both directions.

2. Address Bus

The address bus carries the address of the memory location or I/O device that the CPU wants to access. It is typically unidirectional, from the CPU to other components.

3. Control Bus

The control bus carries control and timing signals that coordinate computer operations, such as memory read, memory write, and interrupt signals. Its signals may travel in either direction.

Memory organization

Memory chips are organized into a number of locations within the IC. Each location can hold 1 bit, 4 bits, 8 bits, or even 16 bits, depending on how it is designed internally. The number of bits that each location within the memory chip can hold is always equal to the number of data pins on the chip. How many locations exist inside a memory chip? That depends on the number of address pins. The number of locations within a memory IC always equals 2 to the power of the number of address pins. Therefore, the total number of bits that a memory chip can store is equal to the number of locations times the number of data bits per location.

To summarize:
1. A memory chip contains 2^x locations, where x is the number of address pins.
2. Each location contains y bits, where y is the number of data pins on the chip.
3. The entire chip will contain $2^x \times y$ bits, where x is the number of address pins and y is the number of data pins on the chip.

A given memory chip has 12 address pins and 4 data pins. Find:
(a) the organization, and (b) the capacity.

Solution:

(a) This memory chip has 4,096 locations ($2^{12} = 4{,}096$), and each location can hold 4 bits of data. This gives an organization of $4{,}096 \times 4$, often represented as $4K \times 4$.
(b) The capacity is equal to 16K bits since there is a total of 4K locations and each location can hold 4 bits of data.

A 512K memory chip has 8 pins for data. Find:
(a) the organization, and (b) the number of address pins for this memory chip.

Solution:

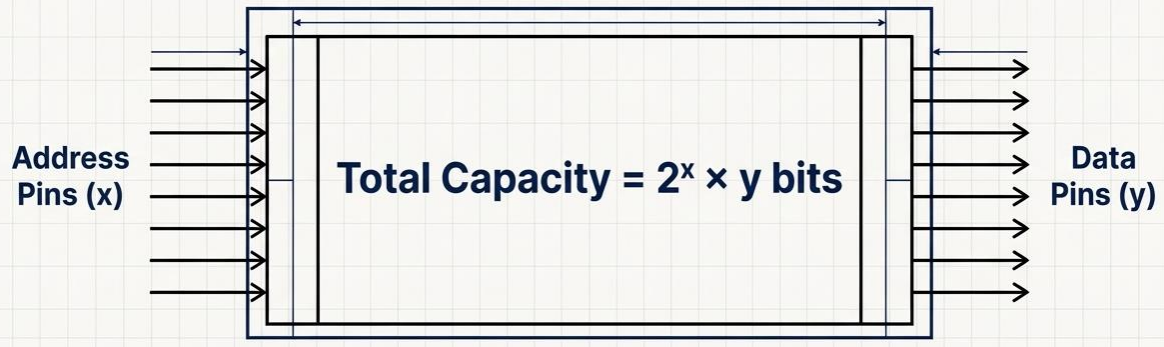
(a) A memory chip with 8 data pins means that each location within the chip can hold 8 bits of data. To find the number of locations within this memory chip, divide the capacity by the number of data pins. 512K/8 = 64K; therefore, the organization for this memory chip is $64K \times 8$.
(b) The chip has 16 address lines since $2^{16} = 64K$.

Internal Memory Chip Organization

Locations: A chip contains 2^x memory locations, where x is the number of address pins.

Capacity per Location: Each location holds y bits, where y is the number of data pins.

Total Capacity Formula: The entire chip stores $2^x \times y$ total bits.



CPU ARCHITECTURE

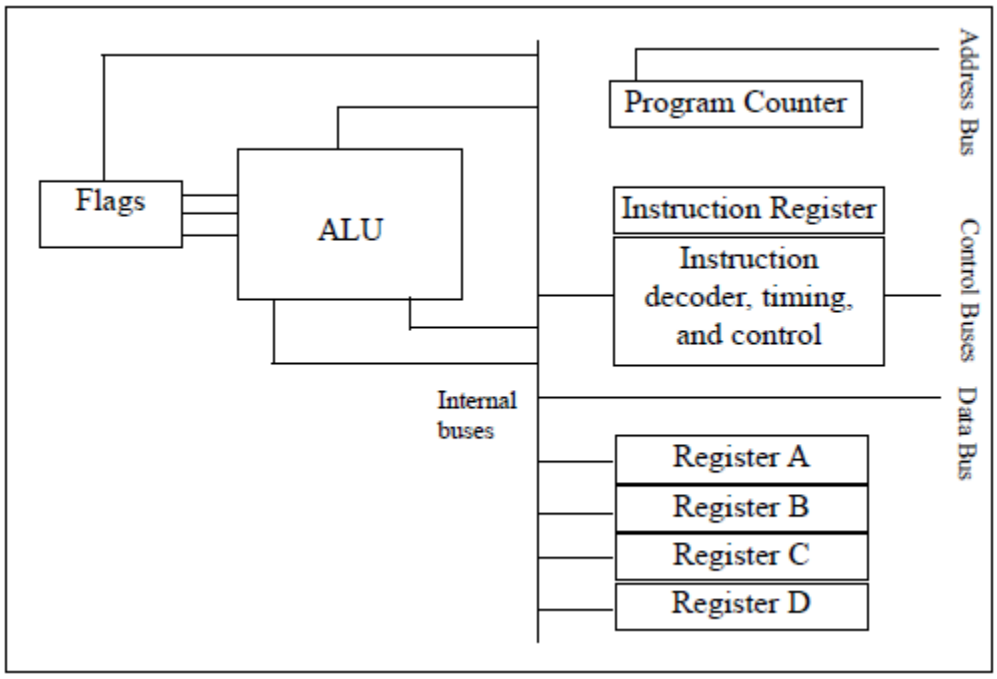
The Central Processing Unit (CPU) is the main processing component of a computer. Its architecture consists of three main parts: the Control Unit (CU), which coordinates operations; the Arithmetic Logic Unit (ALU), which performs arithmetic and logical operations; and registers, which temporarily store data and instructions.

The CPU communicates with memory and input/output devices through the data bus, address bus, and control bus. It executes instructions through the following cycle:

- Fetch: Retrieves an instruction from memory.
- Decode: Interprets the instruction to determine the required operation.
- Execute: Performs the specified operation.

This cycle repeats continuously to execute programs and process data.

Internal block diagram of a CPU



CPU Architecture & The Instruction Cycle

The CPU operates continuously by repeating three steps:

Core Architecture:

- **Control Unit (CU):** Coordinates all system operations.
- **Arithmetic Logic Unit (ALU):** Performs the mathematical and logical operations.
- **Registers:** Provide ultra-fast temporary storage for data and instructions.

The Cycle:

